

Versi 2.0

Januari 2026



BASIS DATA

MODUL 6 - JOIN, SUB QUERY DAN VIEW

Pengarang:

Dr.Ir. Agus Eko Minarno, S.Kom., M.Kom., IPM.

Naufal Arkaan

Ismail Dwi Muh. Anugerah

LABORATORIUM INFORMATIKA

Universitas Muhammadiyah Malang

DAFTAR ISI

DAFTAR ISI.....	2
PENDAHULUAN.....	3
TUJUAN.....	3
TARGET MODUL.....	3
PERSIAPAN.....	3
KATA KUNCI.....	3
BAHAN.....	4
1. SQL Join.....	4
1.1 Inner Join.....	5
1.2 Outer Join.....	6
1.3 Sambungan Silang.....	12
1.4 Sambungan Alami.....	14
2. Subkueri.....	15
2.1 Subkueri Baris Tunggal.....	16
2.2 Subkueri Baris Ganda.....	18
3. Tampilan.....	20
3.1 Buat Tampilan.....	20
3.2 Ganti Tampilan.....	21
3.3 Hapus Tampilan.....	22
3.4 Perbarui Tampilan.....	23
4. Ekspresi Tabel Umum (CTE).....	23
TABEL RINGKASAN	
BAHAN.....	25
KEGIATAN LATIHAN.....	27
SUMBER BELAJAR DAN TUTORIAL.....	28
QUERY DOCK.....	29
TUGAS (DEMO LANGSUNG).....	30
RUBRIK PENILAIAN PRAKTIKUM.....	31



3. Melengkapi mahasiswa dengan keterampilan teoritis dan praktis untuk mengimplementasikan SQL secara efisien. pertanyaan untuk skenario dunia nyata.

3. Siswa mampu menerapkan Ekspresi Tabel Umum (CTE) dan fungsi analitik dasar. seperti RANK dan ROW NUMBER menggunakan Oracle SQL.

yyyyy yy yyy yyy yy yy yy yy yy yy yy . yyy yy yy yyy yyy yyy y y yyy yyy yyy yyy yyy y yy yy y

KEMASYARAKATAN

SQL JOIN, Subquery, View, CTE, Fungsi Analitik, Oracle

BAHAN

1. Gabungan SQL

Terkadang kita perlu menggunakan data dari lebih dari satu tabel untuk membuat laporan. Yang perlu kita lakukan adalah menghubungkan satu tabel dengan tabel lainnya, dan barulah kita dapat mengakses data dari kedua tabel tersebut. Misalkan dalam sistem basis data akademik, data tidak disimpan dalam satu tabel besar, tetapi dipisahkan menjadi beberapa tabel sesuai fungsinya. Dalam skema mahasiswa yang digunakan dalam praktikum ini, data dibagi menjadi:

- **JURUSAN** y menyimpan data jurusan
- **MAHASISWA** y menyimpan data mahasiswa
- **DOSEN** y simpan data dosen
- **KURSUS** y simpan data kursus

Setiap tabel saling terhubung, tetapi **tidak independen**. Misalnya:

1. Data mahasiswa **tidak menyimpan nama departemen**, hanya **ID Jurusan**.
2. Nama jurusan tersimpan dalam tabel **MAJORS**.
3. Untuk menampilkan nama mahasiswa + nama departemen, kedua tabel tersebut harus

Digabungkan.

Proses penggabungan ini disebut **JOIN**.

JOIN adalah operasi dalam SQL yang digunakan **untuk menggabungkan baris data dari dua tabel atau lebih berdasarkan kolom yang terkait**, sehingga menghasilkan informasi yang lebih lengkap dan bermakna.

Contoh:

STUDENTS						MAJORS	
NIM	FULLNAME	MAJORID	GPA	ENROLLMENTYEAR	STATUS	MAJORID	MAJORNAME
2024001	Andi Nugraha	101	3.85	2024	Active	102	Sistem Informasi
2022030	Eka Fitriani	101	3.1	2022	Active	103	Teknik Elektro
2023020	Doni Setiawan	103	3.55	2023	Active	101	Informatika

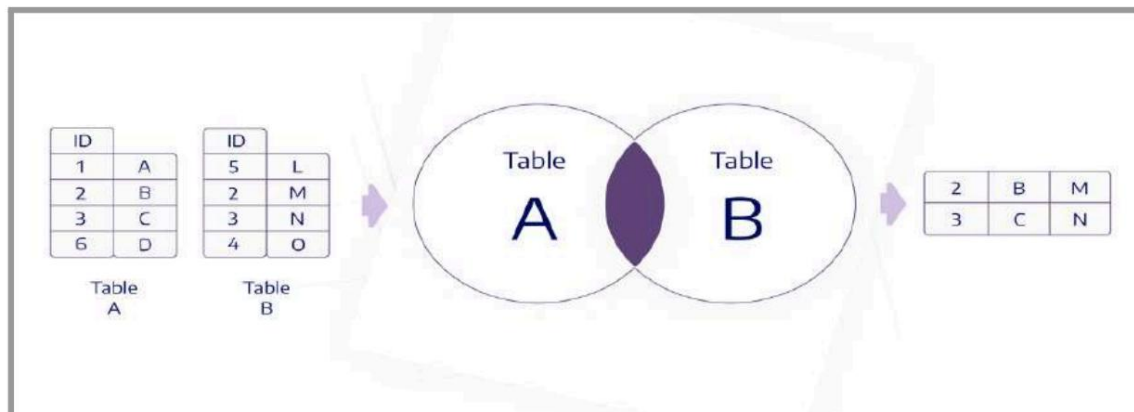


Hasil Penggabungan:

JOIN RESULT			
NIM	FullName	MajorID	MajorName
2024001	Andi Nugraha	101	Informatika
2022030	Eka Fitriani	101	Informatika
2023020	Doni Setiawan	103	Teknik Elektro

Pada contoh di atas, tabel **STUDENTS** berhubungan dengan tabel **MAJORS**. Kolom **MajorID** pada tabel **MAJORS** bertindak sebagai **Kunci Utama**, sedangkan kolom **MajorID** pada tabel **STUDENTS** bertindak sebagai **Kunci Asing** yang menghubungkan data mahasiswa dengan jurusan yang diambil mahasiswa tersebut.

1.1 Penggabungan Dalam (Inner Join)



INNER JOIN adalah jenis operasi JOIN dalam SQL yang digunakan untuk **menggabungkan data dari dua tabel atau lebih berdasarkan kolom yang terkait**. **INNER JOIN** hanya akan **menampilkan baris data yang memiliki nilai yang cocok di kedua tabel** sesuai dengan kondisi **JOIN** yang diberikan.

Dengan kata lain, jika sebuah baris di tabel pertama **tidak memiliki padanan** di tabel kedua, **baris tersebut tidak akan ditampilkan** dalam hasil kueri.

Dalam skema mahasiswa, **INNER JOIN** sering digunakan untuk menampilkan:

- mahasiswa dan jurusan mereka, • mata kuliah dan dosennya, • atau informasi akademik terkait lainnya.



Contoh:

Tampilkan **NIM**, **FullName**, **MajorID**, **MajorName** dengan menggabungkan tabel **STUDENTS** dan **MAJORS** .

```
SELECT S.NIM, S.FullName, S.MajorID, M.MajorName FROM
STUDENTS S INNER
JOIN MAJORS M ON S.MajorID
= M.MajorID;
```

Keluaran:

NIM	FULLNAME	MAJORID	MAJORNAME
2024001	Andi Nugraha	101	Informatika
2022050	Eka Fitriani	101	Informatika
2023020	Doni Setiawan	103	Teknik Elektro
2023015	Budi Cahyono	101	Informatika
2024002	Citra Dewi	102	Sistem Informasi

Penjelasan:

Hasil dari perintah **INNER JOIN** di atas adalah kombinasi data dari tabel **STUDENTS** dan **MAJORS** berdasarkan kolom **MajorID** . Setiap baris hasil menampilkan data mahasiswa yang **MajorID-nya** cocok dengan data jurusan. Nomor ID mahasiswa dan informasi nama diambil dari tabel **STUDENTS** , sedangkan nama jurusan diambil dari tabel **MAJORS** . Mahasiswa yang tidak memiliki **MajorID** yang cocok di tabel **MAJORS** tidak akan ditampilkan dalam hasil **INNER JOIN** .

1.2 Gabungan Luar

OUTER JOIN adalah jenis **JOIN** dalam SQL yang digunakan untuk menggabungkan data dari dua tabel yang saling terkait, tetapi tetap menampilkan data dari satu atau kedua tabel meskipun tidak ditemukan pasangan yang cocok di tabel lainnya. Tidak seperti **INNER JOIN**, yang hanya menampilkan data yang cocok, **OUTER JOIN** memungkinkan data muncul dengan nilai **NULL** di kolom yang sesuai yang tidak memiliki pasangan.

Dalam skema mahasiswa, **OUTER JOIN** berguna untuk menampilkan semua data mahasiswa atau semua data departemen, termasuk data yang belum memiliki hubungan satu sama lain.

OUTER JOIN terdiri dari tiga jenis, yaitu **LEFT OUTER JOIN**, **RIGHT OUTER JOIN**, dan **FULL OUTER JOIN**, yang masing-masing menentukan sisi tabel mana yang tetap ditampilkan meskipun tidak memiliki pasangan data.



• Left Outer Join



LEFT OUTER JOIN adalah jenis operasi **JOIN** dalam SQL yang menampilkan **semua data dari tabel kiri** serta data dari tabel kanan yang sesuai dengan kondisi **JOIN**. Jika ada data di tabel kiri yang **tidak memiliki pasangan** di tabel kanan, data tersebut tetap ditampilkan dengan nilai **NULL** di kolom tabel kanan.

Contoh:

Dalam contoh ini, kita akan menambahkan data ke tabel **LECTURERS** dengan **LecturerID** yang tidak ada di tabel **COURSES** sehingga kita dapat melihat nilai **NULL** :

```
INSERT INTO LECTURERS (LecturerID, LecturerName, Email, MajorID)
NILAI (5004, 'Dr. Rudi Hartono', 'rudi.h@univ.ac.id', 102);
```



Kueri Left Outer Join:

```
SELECT L.LecturerName, C.CourseName FROM
LECTURERS L LEFT
OUTER JOIN COURSES C ON
L.LecturerID = C.LecturerID;
```

Output (Lihat kolom **COURSENAME** di bagian bawah dengan nilai **NULL**):

LECTURERNAME	COURSENAME
Prof. Cahyo Wibowo	Basis Data
Ir. Siti Atsyah	Analisis Sistem
Prof. Cahyo Wibowo	Jaringan Komputer
Dr. Budi Santoso	Pemrograman Dasar
Dr. Rudi Hartono	-

Penjelasan:

LEFT OUTER JOIN di atas menampilkan semua data dosen dari tabel **LECTURERS** sebagai tabel utama. Dosen yang mengajar mata kuliah ditampilkan beserta nama mata kuliahnya, sedangkan dosen yang tidak mengajar mata kuliah apa pun tetap ditampilkan dengan nilai **NULL** di kolom CourseName. Kondisi ini menunjukkan bahwa **LEFT OUTER JOIN** mempertahankan semua data dari tabel sebelah kiri meskipun tidak ada pasangan data di tabel yang tepat.



- Right Outer Join



RIGHT OUTER JOIN adalah jenis operasi JOIN dalam SQL yang **menampilkan semua data dari tabel kanan**, serta data dari tabel kiri yang sesuai dengan kondisi **JOIN**. Jika ada data di tabel kanan yang **tidak memiliki pasangan** di tabel kiri, data tersebut tetap ditampilkan dengan nilai **NULL** di kolom tabel kiri.

Contoh:

Tambahkan 1 Mata Kuliah TANPA Dosen, sehingga kita dapat melihat perbedaan antara right outer join.

```
INSERT INTO COURSES (CourseCode, CourseName, Credits, LecturerID)
VALUES ('IF202', 'Kecerdasan Buatan', 3, NULL);
```

Kueri Right Outer Join:

```
SELECT L.LecturerName, C.CourseName FROM
LECTURERS L RIGHT
OUTER JOIN COURSES C ON
L.LecturerID = C.LecturerID;
```



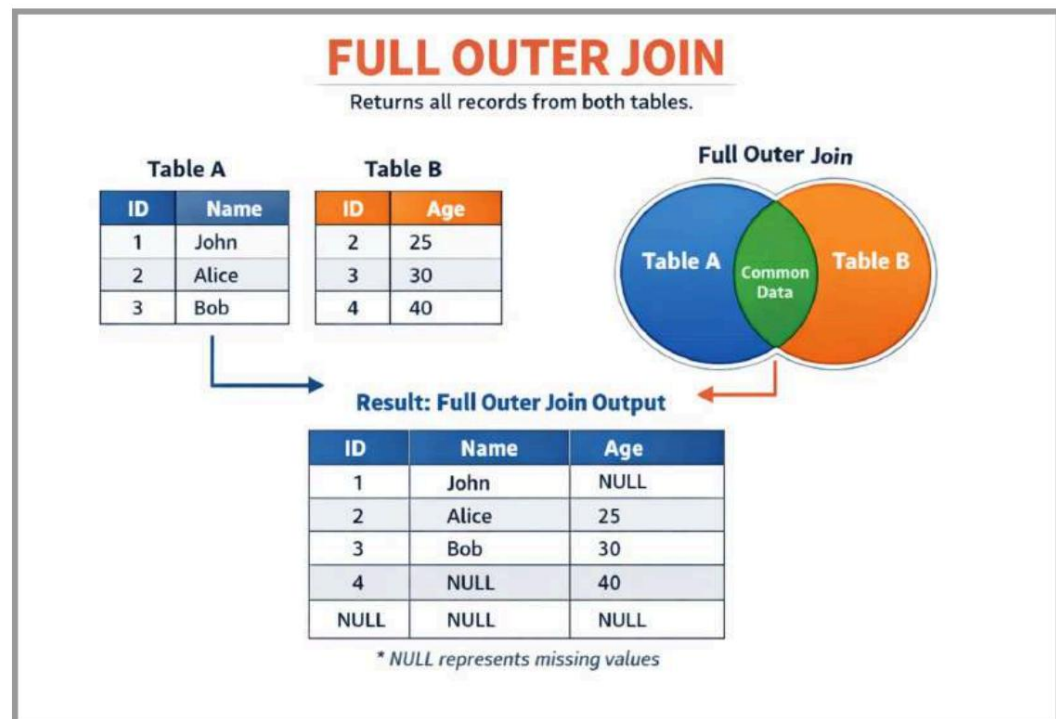
Output (Lihat kolom **LECTURERNAME** paling bawah, **NULL**):

LECTURERNAME	COURSENAME
Prof. Cahyo Wibowo	Basis Data
Prof. Cahyo Wibowo	Jaringan Komputer
Dr. Budi Santoso	Pemrograman Dasar
Ir. Siti Alsyah	Analisis Sistem
-	Kecerdasan Buatan

Penjelasan:

RIGHT OUTER JOIN di atas menampilkan semua data mata kuliah dari tabel **COURSES** sebagai tabel utama. Mata kuliah yang sudah memiliki dosen pengajar akan ditampilkan beserta nama dosennya, sedangkan mata kuliah yang belum memiliki dosen pengajar tetap akan ditampilkan dengan nilai **NULL** pada kolom LecturerName. Kondisi ini dapat dianalogikan sebagai daftar semua mata kuliah yang telah disusun oleh program studi, meskipun tidak semuanya memiliki dosen pengajar yang ditugaskan untuk mengajarnya.

• Full Outer Join



FULL OUTER JOIN adalah jenis operasi JOIN dalam SQL yang **menampilkan semua data dari kedua tabel yang digabungkan**, termasuk data yang cocok dan data yang tidak cocok. Jika sebuah baris dalam satu tabel tidak memiliki kecocokan di tabel lain, kolom dari tabel yang tidak memiliki kecocokan akan memiliki **nilai NULL**.



Sebuah **FULL OUTER JOIN** dapat dipahami sebagai **kombinasi dari LEFT OUTER JOIN dan RIGHT OUTER JOIN**, karena mempertahankan semua data dari kedua sisi tabel.

Contoh:

Menampilkan nama-nama dosen dan nama mata kuliah, termasuk: • dosen yang belum mengajar mata kuliah tersebut, dan • mata kuliah yang belum memiliki dosen yang ditugaskan.

```
SELECT L.LecturerName, C.CourseName FROM
LECTURERS L FULL
OUTER JOIN COURSES C ON
L.LecturerID = C.LecturerID;
```

Keluaran:

LECTURERNAME	COURSENAME
Prof. Cahyo Wibowo	Basis Data
-	Kecerdasan Buatan
Ir. Siti Aisyah	Analisis Sistem
Prof. Cahyo Wibowo	Jaringan Komputer
Dr. Budi Santoso	Pemrograman Dasar
Dr. Rudi Hartono	-

Deskripsi: •

- Dr. Rudi Hartono adalah dosen yang belum pernah mengajar mata kuliah apa pun.
- Kecerdasan Buatan adalah mata kuliah yang belum memiliki dosen.

Penjelasan:

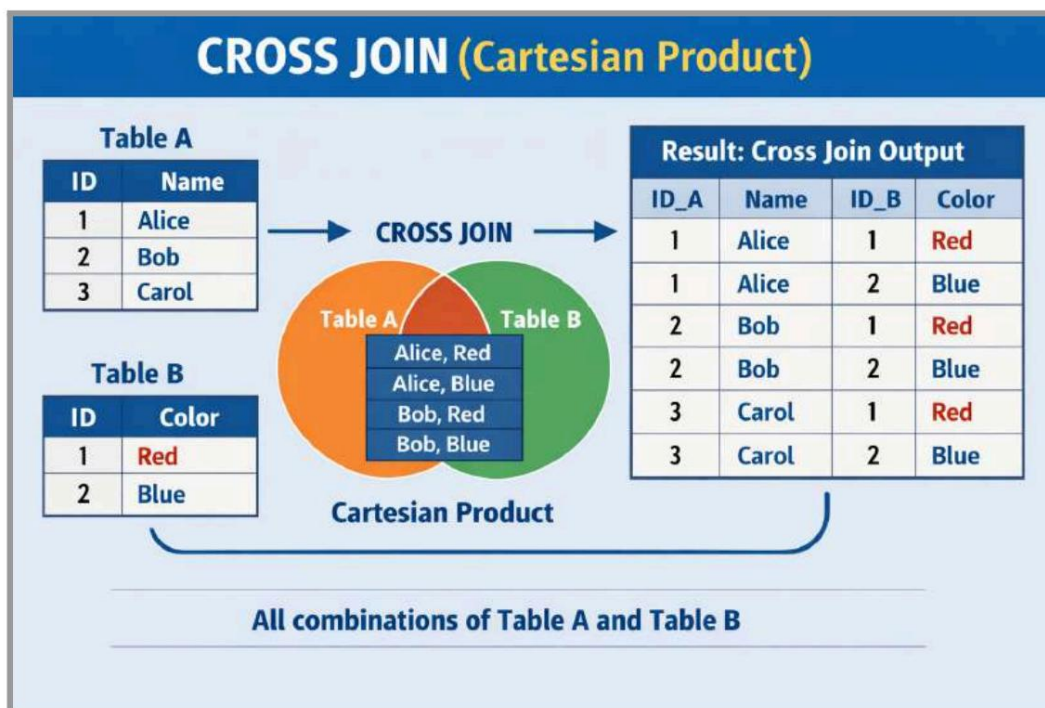
Hasil dari **FULL OUTER JOIN** di atas menampilkan semua data dari tabel **LECTURERS** dan semua data dari tabel **COURSES**. Dosen yang mengajar

Mata kuliah akan ditampilkan beserta nama mata kuliahnya, sedangkan dosen yang tidak mengajar akan tetap ditampilkan dengan nilai **NULL** di kolom CourseName.

Sebaliknya, mata kuliah yang belum memiliki dosen yang ditugaskan juga akan ditampilkan dengan nilai **NULL** pada kolom LecturerName. Kondisi ini dapat dianalogikan sebagai daftar lengkap dosen dan daftar lengkap mata kuliah dalam suatu program studi, di mana tidak semua dosen telah diberi jadwal mengajar dan tidak semua mata kuliah telah ditugaskan dosen.



1.3 Sambungan Silang



CROSS JOIN adalah operasi **JOIN** dalam SQL yang menghasilkan **semua kemungkinan kombinasi baris** dari dua tabel yang digabungkan. **CROSS JOIN** tidak menggunakan kondisi gabungan (**ON**), sehingga setiap baris dalam tabel pertama akan dipasangkan dengan setiap baris dalam tabel kedua. Hasil dari **CROSS JOIN** sering disebut sebagai **produk Cartesian**.

Contoh:

Menampilkan **nama siswa dan nama departemen**, menggabungkan semua siswa dengan semua departemen.

```
SELECT S.FullName, M.MajorName FROM
STUDENTS S CROSS
JOIN MAJORS M;
```

Output:

Output sebenarnya berjumlah total **15 baris** karena tabel **STUDENTS** memiliki **5 baris** x tabel **MAJORS** memiliki **3 baris**, tetapi Oracle APEX **hanya dapat menampilkan 10 baris**. Untuk melihat baris lengkapnya, silakan klik **unduh** di bagian bawah output tabel.



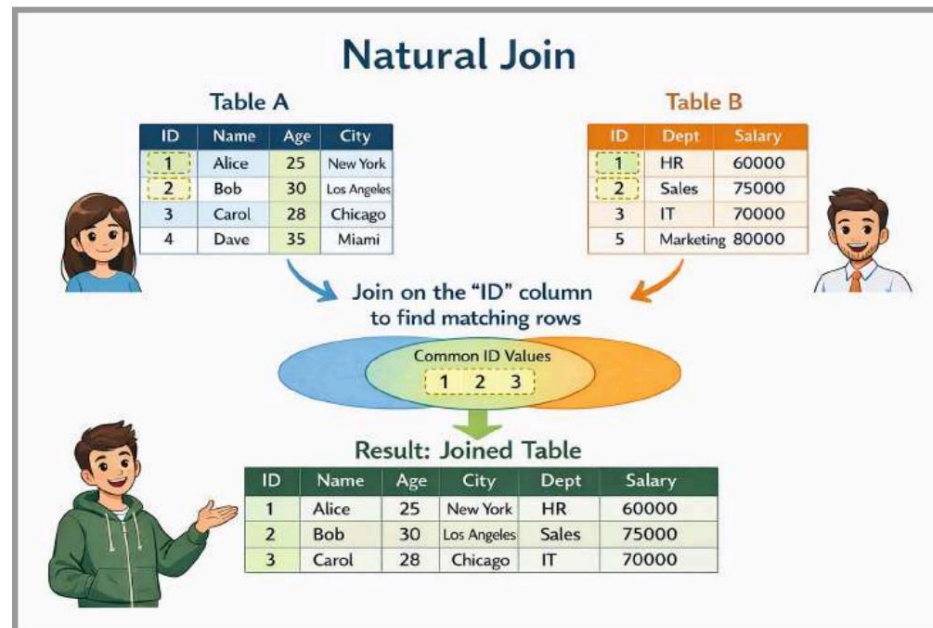
NAMA LENGKAP	NAMA UTAMA
Andi Nugraha	Informatika
Andi Nugraha	Sistem Informasi
Andi Nugraha	Teknik Elektro
Eka Fitriani	Informatika
Eka Fitriani	Sistem Informasi
Eka Fitriani	Teknik Elektro
Doni Setiawan	Informatika
Doni Setiawan	Sistem Informasi
Doni Setiawan	Teknik Elektro
Budi Cahyono	Informatika
Budi Cahyono	Sistem Informasi
Budi Cahyono	Teknik Elektro
Citra Dewi	Informatika
Citra Dewi	Sistem Informasi
Citra Dewi	Teknik Elektro

Penjelasan:

Hasil dari **CROSS JOIN** di atas adalah kombinasi dari semua baris dari tabel MAHASISWA dengan semua baris dari tabel **JURUSAN** tanpa memperhatikan hubungan antar kelompok. Setiap siswa akan dipasangkan dengan setiap jurusan, Meskipun pada kenyataannya mahasiswa tersebut hanya terdaftar di satu jurusan. Kondisi ini dapat dapat dianalogikan sebagai simulasi menempatkan semua siswa di semua jurusan yang tersedia, tanpa mempertimbangkan jurusan asli masing-masing mahasiswa. Oleh karena itu, **GABUNGAN SILANG** jarang digunakan untuk menampilkan data relasional sebenarnya, tetapi sering digunakan untuk simulasi, pengujian sistem, atau untuk keperluan penggabungan data.



1.4 Sambungan Alami



NATURAL JOIN adalah operasi JOIN dalam SQL yang secara otomatis menggabungkan dua tabel berdasarkan kolom yang memiliki nama dan tipe data yang sama. Dalam **NATURAL JOIN**, pengguna **tidak perlu secara eksplisit menulis kondisi JOIN (ON)** karena sistem basis data akan menentukan kolom penghubungnya sendiri.

Contoh:

PILIH NamaLengkap, NamaJurusan
DARI PARA SISWA

BERGABUNG SECARA ALAMI DENGAN JURUSAN UTAMA :

Keluaran:

FULLNAME	MAJORNAME
Andi Nugraha	Informatika
Eka Fitriani	Informatika
Doni Setiawan	Teknik Elektro
Budi Cahyono	Informatika
Citra Dewi	Sistem Informasi

Penjelasan:

Hasil dari penggabungan alami (NATURAL JOIN) di atas adalah kombinasi otomatis dari...
Tabel STUDENTS dan **MAJORS** didasarkan pada kolom **MajorID**, yang memiliki nama dan tipe data yang sama. Setiap mahasiswa akan ditampilkan beserta nama jurusannya tanpa perlu menulis kondisi JOIN.



secara manual. Proses ini dapat dianalogikan sebagai sistem yang mencocokkan data hanya berdasarkan kesamaan nama kolom. Meskipun praktis, **NATURAL JOIN** harus digunakan dengan hati-hati karena dapat menghasilkan hasil yang tidak diinginkan jika terdapat lebih dari satu kolom dengan nama yang sama.

2. Subkueri

Subkueri adalah kueri SQL yang ditulis **di dalam kueri SQL lainnya**. Subkueri digunakan untuk menghasilkan nilai atau kumpulan data yang kemudian **digunakan oleh kueri utama** sebagai dasar untuk pemrosesan lebih lanjut.

Dengan kata lain, subkueri berfungsi sebagai kueri tambahan yang membantu kueri utama dalam mengambil keputusan atau memfilter data. Dalam bidang kemahasiswaan, subkueri sering digunakan untuk menjawab

pertanyaan seperti: A. Mahasiswa dengan IPK tertinggi B. Mahasiswa dengan IPK di atas rata-rata C. Mahasiswa dari jurusan tertentu D. Mata kuliah yang diajarkan oleh dosen tertentu

Beberapa hal penting yang perlu diperhatikan saat menggunakan subkueri:

1. **Subkueri selalu dieksekusi terlebih dahulu** sebelum kueri utama.
2. Subkueri **harus diapit dalam tanda kurung ()**.
3. Subkueri dapat mengembalikan: • satu nilai (subkueri baris tunggal) • lebih dari satu nilai (subkueri multi-baris)

• **Contoh subkueri yang benar (Subkueri baris tunggal):**

Menampilkan siswa dengan **IPK tertinggi**.

```
PILIH NIM, Nama Lengkap, IPK
DARI PARA SISWA
DI MANA IPK = (
  PILIH MAKS(IPK)
  DARI PARA SISWA
);
```

Subkueri:

```
SELECT MAX(GPA)
DARI MAHASISWA
```

Menghasilkan **nilai tunggal**, jadi penggunaan operator = sudah tepat.



Keluaran:

Results Explain Describe Saved SQL History		
NIM	FULLNAME	GPA
2024002	Citra Dewi	3.95
1 rows returned in 0.01 seconds Download		

Contoh Subkueri yang Salah:

```

PILIH NIM, Nama Lengkap, IPK
DARI PARA SISWA
DI MANA IPK = (
  PILIH IPK
  DARI PARA SISWA
);

```

SALAH, karena subkueri mengembalikan **lebih dari satu baris**, sedangkan kueri utama menggunakan operator =, yang hanya dapat membandingkan **satu nilai**.

Penjelasan Subkueri dan Analogi:

Subkueri dapat diibaratkan dengan mencari informasi pendukung sebelum mengambil keputusan besar, seperti mencari tahu IPK tertinggi sebelum menentukan siswa mana yang memiliki IPK tersebut.

2.1 Subkueri Satu Baris

Subkueri satu baris adalah subkueri yang **menghasilkan tepat satu baris (satu nilai)** sebagai hasilnya. Nilai yang dihasilkan oleh subkueri ini kemudian digunakan oleh kueri utama untuk melakukan perbandingan menggunakan operator seperti =, >, <, >=, atau <=.

Subkueri baris tunggal umumnya digunakan ketika hasil yang diinginkan bersifat **unik**, seperti nilai maksimum, nilai minimum, atau hasil perhitungan agregat tertentu.



Contoh:

Menampilkan siswa dengan IPK terendah.

```
PILIH NIM, Nama Lengkap, IPK
DARI PARA SISWA
DI MANA IPK = (
  PILIH MIN(IPK)
  DARI PARA SISWA
);
```

Keluaran:

Results Explain Describe Saved SQL History		
NIM	FULLNAME	GPA
2022030	Eka Fitriani	3.1
1 rows returned in 0.01 seconds Download		

Menghasilkan **nilai IPK terendah**, jadi aman digunakan dengan operator =.

Contoh lain:

Menampilkan **nama-nama mahasiswa dengan IPK tertinggi di jurusan Informatika**.

```
PILIH NIM, Nama Lengkap, IPK
DARI PARA SISWA
DI MANA IPK = (
  PILIH MAKS(IPK)
  DARI PARA SISWA
DI MANA MajorID = (
  PILIH MajorID
  DARI JURUSAN
  WHERE Nama Besar = 'Informatika' ) );
```

Kueri ini digunakan untuk menampilkan mahasiswa dengan IPK tertinggi di jurusan Ilmu Komputer. Subkueri pertama menentukan MajorID **untuk** jurusan Ilmu Komputer dari tabel **MAJORS**. Subkueri berikutnya menghitung IPK tertinggi di jurusan tersebut. IPK tertinggi kemudian digunakan oleh kueri utama untuk menampilkan data mahasiswa yang memiliki IPK tersebut.



Keluaran:

Results Explain Describe Saved SQL History		
NIM	FULLNAME	GPA
2024001	Andi Nugraha	3.85
1 rows returned in 0.02 seconds Download		

Penjelasan & Analogi:

Subkueri baris tunggal digunakan untuk mendapatkan **nilai referensi tunggal** yang kemudian digunakan oleh kueri utama dalam proses penyaringan data. Subkueri dieksekusi terlebih dahulu untuk menentukan nilainya, kemudian kueri utama menggunakan hasilnya untuk menampilkan data yang sesuai. Analogi dari konsep ini adalah seperti menentukan tolok ukur tertentu, misalnya, nilai tertinggi, kemudian menggunakan tolok ukur tersebut untuk memilih data yang memenuhi kriteria.

2.2 Subkueri Multi-baris

Subkueri multi-baris adalah subkueri yang **menghasilkan lebih dari satu baris (beberapa nilai)** sebagai hasilnya. Karena hasilnya lebih dari satu nilai, kueri utama **tidak dapat** menggunakan operator `=`. Sebagai gantinya, operator khusus seperti **IN**, **ANY**, atau **ALL** digunakan.

Subkueri multi-baris cocok digunakan ketika kondisi pencarian **melibatkan banyak data**, seperti mahasiswa dari beberapa departemen atau program studi dari beberapa dosen.

Contoh:

Menampilkan mahasiswa dari jurusan Ilmu Komputer dan Sistem Informasi.

```
PILIH NIM, Nama Lengkap, ID Jurusan
DARI PARA SISWA
DI MANA MajorID DALAM (
  PILIH MajorID
  DARI JURUSAN
  WHERE MajorName IN ('Informatika', 'Sistem Informasi'));
```

Subkueri tersebut menghasilkan **lebih dari satu MajorID**, sehingga operator **IN** digunakan di kueri utama pertanyaan.



Keluaran:

NIM	FULLNAME	MAJORID
2024001	Andi Nugraha	101
2022030	Eka Fitriani	101
2023015	Budi Cahyono	101
2024002	Citra Dewi	102

Contoh lain:

Menampilkan mata kuliah yang diajarkan oleh dosen dari departemen Informatika.

```

PILIH Kode Kursus, Nama Kursus
DARI KURSUS
DI MANA ID Dosen (
  PILIH ID Dosen
  DARI DOSEN
DI MANA MajorID = (
  PILIH MajorID
  DARI JURUSAN
WHERE Nama Besar = 'Informatika' ) );

```

Garis besar singkat:

- Subkueri terdalam mencari MajorID **dari departemen Informatika**.
- Subkueri kedua mengambil **semua LecturerID** dari departemen tersebut.
- Kueri utama menampilkan mata **kuliah yang diajarkan oleh para dosen tersebut**.

Karena subkueri menghasilkan **lebih dari satu LecturerID**, operator **IN** digunakan.

Keluaran:

COURSECODE	COURSENAME
IF102	Basis Data
EL305	Jaringan Komputer
IF101	Pemrograman Dasar



Penjelasan & Analogi:

Subkueri multi-baris digunakan ketika suatu kondisi tunggal membutuhkan **lebih dari satu** nilai referensi. Subkueri dieksekusi terlebih dahulu untuk menghasilkan daftar nilai, kemudian kueri utama mencocokkan data dengan daftar tersebut. Analogi untuk subkueri multi-baris adalah seperti membuat daftar kriteria terlebih dahulu, misalnya, beberapa jurusan atau beberapa dosen, kemudian memilih data yang termasuk dalam daftar tersebut.

3. Lihat

VIEW adalah **tabel virtual** yang dibuat dari hasil kueri SQL. VIEW tidak menyimpan data secara fisik, tetapi hanya menyimpan **definisi kueri** yang akan dieksekusi setiap kali VIEW dipanggil. Data yang ditampilkan oleh VIEW selalu mengikuti data terbaru dalam tabel sumber.

VIEW digunakan untuk menyederhanakan kueri yang kompleks, meningkatkan keamanan data, dan memudahkan pengguna mengakses informasi tertentu tanpa harus mengetahui struktur tabel secara detail.

3.1 Buat Tampilan

CREATE VIEW adalah perintah SQL yang digunakan untuk **membuat VIEW**, yaitu tabel virtual yang berasal dari hasil kueri. VIEW menyimpan definisi kueri, bukan data fisik, sehingga data yang ditampilkan oleh VIEW selalu sesuai dengan data terbaru dalam tabel sumbernya.

Struktur Dasar dari CREATE VIEW:

```
BUAT VIEW view_name SEBAGAI
SELECT column1, column2, ...
DARI nama_tabel
WHERE kondisi;
```

Deskripsi: •

view_name: nama VIEW yang akan dibuat • **SELECT:** kueri yang menjadi dasar VIEW • **WHERE:** kondisi (opsional)

Contoh Tampilan: Data Mahasiswa dan Jurusan

Buat TAMPILAN untuk menampilkan **nomor ID mahasiswa, nama mahasiswa, dan nama departemen**.

```
BUAT VIEW v_mahasiswa_jurusan SEBAGAI
SELECT S.NIM, S.FullName, M.MajorName FROM
STUDENTS S
```



```
JOIN MAJORS M ON
S.MajorID = M.MajorID;
```

Setelah VIEW dibuat, data dapat ditampilkan dengan perintah:

```
PILIH * DARI v_mahasiswa_jurusan;
```

NIM	FULLNAME	MAJORNAME
2024001	Andi Nugraha	Informatika
2022030	Eka Fitriani	Informatika
2023020	Doni Setiawan	Teknik Elektro
2023015	Budi Cahyono	Informatika
2024002	Citra Dewi	Sistem Informasi

VIEW **v_mahasiswa_jurusan** akan menampilkan data mahasiswa beserta jurusan mereka. tanpa perlu menulis **JOIN** berulang kali.

3.2 Tampilan Pengganti

CREATE OR REPLACE VIEW adalah perintah SQL yang digunakan untuk **mengubah atau memperbarui definisi VIEW yang sudah ada** tanpa harus menghapus VIEW tersebut terlebih dahulu. Jika VIEW belum ada, perintah ini akan membuat VIEW baru. Jika VIEW sudah ada, definisinya akan diganti dengan yang baru.

Struktur Dasar dari CREATE OR REPLACE VIEW:

```
BUAT ATAU GANTI TAMPILAN view_name
SEBAGAI PILIH kolom1, kolom2, ...
DARI nama_tabel
WHERE kondisi;
```

Deskripsi: •

view_name: nama VIEW yang akan dibuat atau diperbarui • **SELECT:** kueri baru yang menjadi definisi VIEW • **WHERE:** kondisi (opsional)



Contoh CREATE OR REPLACE VIEW: Contoh:

Memperbarui Tampilan Mahasiswa dan Departemen Misalkan

VIEW `v_mahasiswa_jurusan` sudah ada.

Sekarang, TAMPILAN diperbarui dengan menambahkan kolom **IPK** .

```
BUAT ATAU GANTI VIEW v_mahasiswa_jurusan SEBAGAI SELECT
S.NIM, S.FullName, S.GPA, M.MajorName FROM STUDENTS
S JOIN MAJORS M ON
S.MajorID = M.MajorID;
```

Setelah diganti, VIEW dapat dipanggil lagi dengan:

```
PILIH * DARI v_mahasiswa_jurusan;
```

NIM	FULLNAME	GPA	MAJORNAME
2024001	Andi Nugraha	3.85	Informatika
2022030	Eka Fitriani	3.1	Informatika
2023020	Doni Setiawan	3.55	Teknik Elektro
2023015	Budi Cahyono	3.2	Informatika
2024002	Citra Dewi	3.95	Sistem Informasi

Perintah ini akan menampilkan data siswa beserta **jurusan dan IPK** mereka.
sesuai dengan definisi VIEW terbaru.

3.3 Tampilan Jatuh

DROP VIEW adalah perintah SQL yang digunakan untuk **menghapus VIEW** dari sebuah database.

Perintah ini hanya menghapus **definisi VIEW**, bukan data dalam tabel sumber. Setelah sebuah VIEW dihapus, VIEW tersebut tidak dapat digunakan lagi kecuali dibuat ulang dengan perintah **CREATE VIEW** .

Struktur Dasar Tampilan Tarik-Tarik (DROP VIEW):

```
TARIK TUTUP view_name;
```



3.4 Tampilan Pembaruan

UPDATE VIEW adalah perintah SQL yang digunakan untuk **memperbarui data melalui sebuah VIEW**. Perubahan yang dilakukan pada VIEW sebenarnya **secara langsung memengaruhi data dalam tabel sumber**, selama VIEW tersebut memenuhi persyaratan untuk **view yang dapat diperbarui** (misalnya, berasal dari satu tabel utama dan tidak menggunakan fungsi agregasi).

Struktur Dasar PERBARUI TAMPILAN:

```
PERBARUI nama_tampilan
SET column_name = new_value
Kondisi DI MANA ;
```

Deskripsi: •

view_name: nama VIEW yang akan dibuat atau diperbarui • **SELECT:** kueri baru yang menjadi definisi VIEW • **WHERE:** kondisi (opsional)

Contoh dari UPDATE VIEW:

Misalkan terdapat TAMPILAN berikut:

```
BUAT LIHAT v_data_mahasiswa AS
PILIH NIM, Nama Lengkap, IPK, Status
DARI PARA SISWA;
```

Contoh Pembaruan Data melalui VIEW

Mengubah IPK mahasiswa dengan nomor identitas mahasiswa 2024001:

```
UPDATE v_data_mahasiswa SET
GPA = 3.90
WHERE NIM = '2024001';
```

Perintah ini akan **mengubah nilai IPK di tabel MAHASISWA**, meskipun perintah UPDATE dieksekusi melalui VIEW.

4. Ekspresi Tabel Umum (CTE)

Common Table Expressions (CTE) adalah fitur SQL yang digunakan untuk **membuat hasil kueri sementara yang dapat dipanggil kembali dalam kueri utama**. CTE ditulis menggunakan klausa **WITH** dan bertujuan untuk membuat kueri lebih rapi, lebih mudah dibaca, dan lebih mudah dipahami, terutama ketika kueri memiliki logika yang kompleks.



CTE bersifat sementara dan **hanya berlaku selama kueri berjalan**.

Contoh Permintaan Informasi CTE (Skema Mahasiswa)

Menampilkan siswa dengan IPK di atas IPK rata-rata semua siswa.

```
DENGAN RataIPK SEBAGAI (  
  SELECT AVG(GPA) AS AvgGPA  
  DARI PARA SISWA  
)  
PILIH NIM, Nama Lengkap, IPK  
DARI PARA SISWA  
WHERE GPA > (SELECT AvgGPA FROM RataIPK);
```

Penjelasan & Analogi:

Dalam contoh di atas, CTE digunakan untuk menghitung **rata-rata IPK** siswa dan menyimpannya sementara dengan nama **RataIPK**. Nilai tersebut kemudian digunakan oleh kueri utama untuk menampilkan siswa yang memiliki IPK di atas rata-rata. CTE dapat dianalogikan sebagai mencatat hasil perhitungan penting dalam catatan sementara, kemudian menggunakan catatan tersebut sebagai referensi dalam pengambilan keputusan selanjutnya.



TABEL RINGKASAN BAHAN

Pertanyaan	Cara Membaca Bahasa Indonesia (Analogi Singkat)
SELECT S.FullName, M.MajorName FROM STUDENTS S INNER JOIN MAJORS M ON S.MajorID = M.MajorID;	Ambil nama mahasiswa dan nama jurusannya, hanya untuk mahasiswa yang jurusannya benar-benar ada dan cocok.
SELECT L.LecturerName, C.CourseName FROM LECTURERS L LEFT JOIN COURSES C ON L.LecturerID = C.LecturerID;	Tampilkan semua dosen, meskipun ada dosen yang belum mengajar mata kuliah.
SELECT L.LecturerName, C.CourseName FROM LECTURERS L RIGHT JOIN COURSES C ON L.LecturerID = C.LecturerID;	Tampilkan semua mata kuliah, meskipun ada mata kuliah yang belum punya dosen.
SELECT L.LecturerName, C.CourseName FROM LECTURERS L FULL JOIN COURSES C ON L.LecturerID = C.LecturerID;	Tampilkan semua dosen dan semua mata kuliah , baik yang saling berhubungan maupun yang belum.
SELECT S.FullName, M.MajorName FROM STUDENTS S CROSS JOIN MAJORS M;	Pasangkan setiap siswa dengan semua jurusan , tanpa peduli jurusan yang sebenarnya.
SELECT FullName, MajorName FROM MAHASISWA SECARA ALAMI BERGABUNG DENGAN JURUSAN UTAMA;	Gabungkan mahasiswa dan arahkan secara otomatis berdasarkan kolom yang namanya sama.
SELECT NIM, FullName, GPA FROM STUDENTS DI MANA IPK = (PILIH MINIMUM(IPK) DARI SISWA);	Tampilkan siswa yang IPK-nya paling rendah.
SELECT CourseCode, CourseName FROM KURSUS DI MANA Kredit = (PILIH MAKSIMUM (Kredit) DARI KURSUS);	Cari mata kuliah dengan jumlah SKS paling besar.
SELECT NIM, FullName FROM STUDENTS DI MANA MajorID DALAM (PILIH MajorID DARI JURUSAN DI MANA Nama Jurusan (‘Informatika’, ‘Sistem Informasi’));	Tampilkan mahasiswa yang berasal dari jurusan Informatika atau Sistem Informasi.
BUAT VIEW v_mahasiswa_jurusan SEBAGAI SELECT S.NIM, S.FullName, M.MajorName FROM STUDENTS S JOIN MAJORS M ON S.MajorID = M.MajorID;	Buat tabel virtual yang menyimpan data mahasiswa beserta jurusannya agar tidak perlu menulis JOIN berulang kali.
UPDATE v_data_mahasiswa SET IPK = 3.90 Ubah IPK mahasiswa tertentu	Ubah IPK mahasiswa tertentu melalui VIEW,



DI MANA NIM = '2024001';	tapi datanya tetap berubah di tabel asli.
DENGAN RataIPK SEBAGAI (PILIH AVG(GPA) SEBAGAI (AvgGPA FROM STUDENTS) SELECT FullName, IPK DARI MAHASISWA DI MANA IPK > (PILIH) Rata-rata IPK (dari RataIPK);	Hitung dulu rata-rata IPK , lalu tampilkan siswa yang IPK-nya di atas rata-rata tersebut .



KEGIATAN PRAKTIK

Dalam kegiatan praktik ini, siswa diminta untuk membuat skema basis data baru dengan tema pilihan mereka berdasarkan kreativitas mereka. Skema tersebut harus logis dan terdiri dari setidaknya dua tabel yang saling terkait. Setelah skema dan data dibuat, siswa diharuskan untuk menerapkan beberapa konsep kueri yang telah mereka pelajari di Modul 6.

Instruksi:

1. Buat **skema basis data baru dengan tema gratis** (misalnya, toko online, perpustakaan, klinik, penyewaan peralatan, sistem acara, dll.).
2. Pastikan skema yang dibuat memiliki **relasi antar tabel** yang memungkinkan penerapan **JOIN** dan **subkueri**.
3. Gunakan skema yang telah Anda buat untuk menyusun kueri berikut: • **LEFT JOIN**, untuk menampilkan semua data dari tabel utama meskipun tidak ada kecocokan di tabel lain.
 - **Subkueri baris tunggal**, untuk menampilkan data dengan nilai tertinggi atau terendah berdasarkan suatu kriteria.
 - **CREATE VIEW**, untuk membuat tampilan dari hasil kueri yang melibatkan lebih dari satu tabel.
 - **SELECT from VIEW**, untuk menampilkan data spesifik dari tampilan yang telah dibuat. • **CTE (Common Table Expressions)**, untuk menghitung atau meringkas data yang ditambahkan nilai.
4. Jalankan setiap kueri dan sertakan hasil keluaran yang diperoleh.
5. Hasilnya akan dipresentasikan pada sesi Query Dock.



SUMBER BELAJAR DAN TUTORIAL

[LINK_W3SCHOOLS_SQL-JOIN](#) (Rekomendasi)

[LINK_YOUTUBE_SQL-JOINS](#) (Rekomendasi)

[LINK_DOCS-ORACLE](#)



DOCK PERTANYAAN

Query Dock adalah sesi diskusi dan presentasi yang bertujuan untuk meninjau dan memperdalam pemahaman siswa tentang materi di Modul 6. Dalam sesi ini, siswa akan mempresentasikan hasil **Aktivitas Praktik** yang telah mereka selesaikan sebelumnya dan secara aktif mendiskusikannya dengan asisten laboratorium dan siswa lainnya.

Alur Aktivitas Query Dock: 1.

Presentasi Hasil Aktivitas Praktik Siswa diminta

untuk mempresentasikan: • Skema

basis data yang dibuat (tema dan hubungan antar tabel) • Kueri yang telah dikerjakan meliputi:

- BERGABUNG KE KIRI
- Subkueri satu baris • BUAT VIEW dan PILIH dari VIEW
- CTE
- Hasil keluaran dari setiap kueri 2.

Penjelasan Kueri

Untuk setiap kueri yang disajikan, peserta pelatihan diharapkan mampu:

- Menjelaskan fungsi dan tujuan kueri. • Menyampaikan logika kueri dalam bahasa yang sederhana. • Menjelaskan alasan penggunaan konsep (JOIN, subkueri, VIEW, atau CTE).

3. Sesi Tanya Jawab Antar Peserta Magang

Peserta pelatihan lainnya dipersilakan untuk:

- Ajukan pertanyaan terkait kueri atau skema yang disajikan • Berikan masukan atau pendapat selama sesi diskusi 4. **Pertanyaan dari Asisten**

Laboratorium

Jika para siswa telah memahami materi dan tidak memiliki pertanyaan lebih lanjut, asisten laboratorium akan mengajukan pertanyaan tambahan terkait Modul 6, seperti:

- Perbedaan Antara LEFT JOIN dan INNER JOIN • Perbedaan Antara Subkueri Baris Tunggal dan Baris Ganda • Fungsi VIEW dan Alasan Penggunaannya • Kegunaan CTE Dibandingkan dengan Subkueri



TUGAS (DEMO LANGSUNG)

Live Demo adalah sesi evaluasi langsung yang bertujuan untuk menguji pemahaman peserta pelatihan terhadap materi Modul 6 secara mandiri dan secara real-time. Dalam sesi ini, peserta pelatihan diharuskan mampu menganalisis output tabel dan menyusun kueri SQL yang sesuai tanpa diberikan contoh kueri sebelumnya.

Syarat dan Ketentuan Demo Langsung:

1. Siswa **tidak diberikan kueri SQL**, tetapi hanya **hasil keluaran tabel** yang harus dianalisis.
2. Mahasiswa diharuskan untuk: •
Memahami struktur tabel yang diberikan • Menulis query SQL yang benar sehingga menghasilkan output tabel yang sama seperti yang diberikan.
3. Waktu yang diberikan untuk Demo Langsung bagi setiap siswa adalah maksimal **30 menit**.

Catatan:

Selama sesi Demo Langsung, penilaian tidak hanya berfokus pada hasil kueri akhir, tetapi juga pada **proses berpikir, logika, dan penjelasan** yang diberikan oleh siswa dalam menyusun kueri berdasarkan keluaran tabel yang diberikan.



RUBRIK PENILAIAN PRAKTIKUM

Aspek Penilaian	Poin
Dock Kueri	Total 10%
Tugas (DEMO LANGSUNG)	Total 90%
Pertanyaan 1	5%
Pertanyaan 2	10%
Pertanyaan 3	25%
Pertanyaan 4	25%
Pertanyaan 5	25%

Catatan Penting tentang Pelaksanaan Praktikum

Selama pekan **Query Dock**, peserta pelatihan didorong untuk **mengajukan pertanyaan** tentang modul 6. materi yang belum mereka pahami. Sesi ini tidak hanya menguji sintaksis, tetapi juga membantu Para peserta pelatihan memperdalam pemahaman mereka tentang konsep SQL melalui diskusi dengan para asisten.

Skor Query **Dock memiliki persentase kecil**, sehingga fokus utama penilaian tersebut adalah... Keberhasilan tetap bergantung pada **pemahaman materi** dan kemampuan siswa untuk menjelaskan logika dari pertanyaan tersebut.

Untuk setiap item penilaian pada rubrik:

- 100 ÿ Jawaban yang benar dan percaya diri / penjelasan yang jelas
- 50 ÿ Jawaban sebagian benar / masih ragu
- 0 ÿ Tidak dapat menjawab / tidak mengikuti instruksi

Dengan skema ini, penilaian dilakukan secara objektif dan konsisten sesuai dengan levelnya. dari penguasaan siswa.

